ECOLE DES HAUTES ETUDES D'INGENIERIE -- OUJDA

Mémoire de PFA

FILIERE : GENIE INFORMATIQUE

TITRE

Réalisé par :

M. ALWAN Charaf Eddine
M. KMIMECH Youssef
M. BOUAMAR Ayman
M. NIKEL Mohammed Taha
M. BEDLOU Oussama

Encadré par :

M. MANI Mohammed Adil

Soutenance le ..... 2025 devant le Jury :

M/MME .....

M/MME .....

M/MME .....

Année universitaire : 2025 – 2026

Dédicace

Tout travail académique abouti est le fruit d'un parcours jalonné d'efforts, de remises en question et de soutiens précieux. Ce projet de fin d'année s'inscrit dans cette dynamique : il représente bien plus qu'une simple réalisation technique, mais l'aboutissement d'un chemin parcouru avec persévérance, entraide et détermination. C'est à ce titre que nous souhaitons consacrer ces quelques lignes à celles et ceux qui ont contribué, de manière essentielle, à sa concrétisation.

Nous dédions ce travail avant tout à nos parents, dont le soutien indéfectible a constitué le socle de notre réussite. Par leur patience, leurs encouragements constants et les sacrifices consentis tout au long de notre parcours académique, ils ont su nous offrir un cadre propice à l'apprentissage et à l'épanouissement. Leur confiance et leur présence, tant dans les moments d'incertitude que dans les phases de réussite, ont été une source permanente de motivation et de stabilité.

Nous tenons également à mettre à l'honneur les membres de notre équipe, **THE GANG OF FIVE**. Ce projet n'aurait pu aboutir sans leur sérieux, leur sens des responsabilités et leur engagement collectif. Chacun, par son investissement, sa rigueur et son esprit de collaboration, a contribué à faire de ce travail une réalisation cohérente et aboutie. Les échanges constructifs, la complémentarité des compétences et la capacité à faire face ensemble aux défis rencontrés ont été des éléments déterminants tout au long du projet.

Enfin, nous adressons notre reconnaissance à toutes les personnes qui, de près ou de loin, ont cru en nous et nous ont encouragés à aller au-delà de nos limites. À travers leurs conseils, leur disponibilité et leurs orientations, nos enseignants ont contribué à structurer notre réflexion et à renforcer notre rigueur académique. Nous pensons également à nos camarades et collègues, dont le soutien moral et la présence ont accompagné notre parcours. L'ensemble de ces apports a joué un rôle essentiel dans notre progression, tant sur le plan académique que personnel, et a contribué à donner à ce projet toute sa valeur.

Remerciements

Au terme de ce projet de fin d'année, il nous est particulièrement agréable d'exprimer notre profonde reconnaissance envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Nous tenons tout d'abord à adresser nos remerciements les plus sincères à Monsieur BARBOUCHA Mohammed, Directeur de l'École des Hautes Études d'Ingénierie, pour la vision qu'il porte et qui fait de l'EHEI bien plus qu'un simple établissement d'enseignement : un véritable tremplin vers l'excellence et l'innovation. Le cadre académique stimulant qu'il a su instaurer nous a permis de nous épanouir et de développer notre esprit d'initiative, essence même de ce projet.

Nous exprimons notre gratitude particulière à Monsieur MANI Mohammed Adil, notre encadrant technique, dont l'accompagnement a été un pilier fondamental dans la concrétisation de notre vision. Sa disponibilité constante, ses conseils avisés et sa capacité à nous guider tout en respectant notre autonomie créative ont été déterminants. Au-delà de l'expertise technique, il a su être un véritable mentor et un soutien psychologique précieux. Dans les moments de doute, face à la complexité de concilier documentation, apprentissage autonome de nouvelles technologies et charge académique, ses paroles encourageantes et sa bienveillance nous ont redonné confiance et motivation. Son attention particulière à notre bien-être physique et mental, ainsi que ses orientations méthodologiques, nous ont permis de persévérer et de transformer les obstacles en opportunités d'apprentissage. Sa confiance inébranlable en notre capacité à explorer des domaines non couverts par notre programme d'études nous a encouragés à repousser nos limites et à croire en notre potentiel.

Nos remerciements s'adressent également à notre encadrant pédagogique, dont le suivi rigoureux et les recommandations méthodologiques ont contribué à structurer notre démarche et à maintenir la qualité académique de notre travail tout au long de ce projet d'envergure.

Nous souhaitons remercier chaleureusement l'ensemble du corps professoral et administratif de l'EHEI. Leurs enseignements, qui allient théorie et pratique, nous ont fourni les fondations solides nécessaires pour concevoir et développer une application aussi ambitieuse. Les compétences techniques, mais aussi les valeurs de rigueur, de persévérance et de travail collaboratif qu'ils nous ont transmises, constituent aujourd'hui notre plus précieux bagage pour aborder notre future vie professionnelle.

Enfin, nous tenons à exprimer notre reconnaissance envers l'EHEI dans son ensemble, qui nous a offert bien plus qu'une formation : une communauté d'apprentissage, un environnement

propice à l'innovation et une expérience humaine enrichissante qui restera gravée dans nos mémoires.

Ce projet représente pour nous l'aboutissement de quatre années d'efforts, de découvertes et de croissance personnelle. Il symbolise notre passage de l'apprentissage à la création, de l'étudiant à l'ingénieur. Pour cela, we be eternally grateful.

Résumé

Ce rapport présente le projet de fin d'année mené au sein de la 4ème année **Génie Informatique**. Il s'inscrit dans une démarche visant à mettre en pratique les **compétences** acquises tout au long de notre formation et à nous préparer aux exigences du monde professionnel. L'objectif principal de ce travail est la conception et le développement d'une application destinée à soutenir les étudiants en génie informatique dans leur organisation, leur **apprentissage** et leur préparation à l'**insertion professionnelle**.

L'idée du projet est née de notre propre expérience étudiante et de l'identification de besoins réels liés à **la gestion des projets**, au **développement de compétences techniques**, à la **préparation aux entretiens** et à l'amélioration de **la présentation professionnelle**. Pour y répondre, nous avons imaginé une **plateforme intégrée** regroupant plusieurs outils complémentaires, chacun destiné à accompagner l'étudiant dans une dimension essentielle de son parcours.

Notre démarche s'est articulée autour d'une analyse approfondie des besoins, puis d'une phase de conception structurée permettant de définir l'architecture générale de la solution. Ce rapport retrace l'ensemble du processus suivi, depuis la définition de l'idée initiale jusqu'à la formalisation du concept final, en passant par les choix méthodologiques qui ont guidé notre travail collectif.

Ce projet représente une occasion d'appliquer nos connaissances, mais aussi une expérience collaborative enrichissante, renforçant nos compétences en organisation, communication et conception au sein d'un travail d'équipe.

Mots Clés : génie informatique, compétences, apprentissage, insertion professionnelle, gestion des projets, développement de compétences techniques, préparation aux entretiens, présentation professionnelle, plateforme intégrée.

Abstract

This report presents the final year project conducted within the 4th year of **Computer Engineering**. It is part of an approach aimed at applying the **skills** acquired throughout our training and preparing us for the demands of the professional world. The main objective of this work is the design and development of an application intended to support computer engineering students in their organization, **learning**, and preparation for **professional integration**.

The project idea originated from our own student experience and the identification of real needs related to **project management**, **technical skills development**, **interview preparation**, and improvement of **professional presentation**. To address these needs, we envisioned an **integrated platform** bringing together several complementary tools, each designed to support students in an essential dimension of their academic journey.

Our approach was structured around an in-depth needs analysis, followed by a structured design phase that defined the overall architecture of the solution. This report traces the entire process followed, from the definition of the initial idea to the formalization of the final concept, including the methodological choices that guided our collective work.

This project represents an opportunity to apply our knowledge, but also an enriching collaborative experience, strengthening our skills in organization, communication, and design within teamwork.

Keywords: computer engineering, skills, learning, professional integration, project management, technical skills development, interview preparation, professional presentation, integrated platform.

ملخص

يقدم هذا التقرير مشروع نهاية السنة الذي تم إنجازه في إطار السنة الرابعة من **هندسة المعلوميات**. يندرج هذا المشروع ضمن نهج يهدف إلى تطبيق **الكفاءات** المكتسبة طوال فترة تكويننا وإعدادنا لمتطلبات العالم المهني. الهدف الرئيسي من هذا العمل هو تصميم وتطوير تطبيق يهدف إلى دعم طلاب هندسة المعلوميات في تنظيمهم و**تعلمهم** وإعدادهم **للاندماج المهني**.

نشأت فكرة المشروع من تجربتنا الطلابية الخاصة ومن تحديد الاحتياجات الحقيقية المتعلقة **بإدارة المشاريع**، و**تطوير المهارات التقنية**، و**التحضير للمقابلات**، وتحسين **العرض المهني**. وللإجابة على هذه الاحتياجات، تصورنا **منصة متكاملة** تجمع بين عدة أدوات تكميلية، كل منها مصمم لمرافقة الطالب في بُعد أساسي من مساره الأكاديمي.

تمحور نهجنا حول تحليل معمق للاحتياجات، يليه مرحلة تصميم منظمة سمحت بتحديد الهندسة العامة للحل. يتتبع هذا التقرير العملية برمتها، من تحديد الفكرة الأولية إلى إضفاء الطابع الرسمي على المفهوم النهائي، مرورًا بالاختيارات المنهجية التي وجهت عملنا الجماعي.

يمثل هذا المشروع فرصة لتطبيق معارفنا، ولكنه أيضًا تجربة تعاونية مثرية، تعزز مهاراتنا في التنظيم والتواصل والتصميم ضمن العمل الجماعي.

الكلمات المفتاحية: هندسة المعلوميات، الكفاءات، التعلم، الاندماج المهني، إدارة المشاريع، تطوير المهارات التقنية، التحضير للمقابلات، العرض المهني، منصة متكاملة.

Liste des abréviations

Abréviation	Désignation
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
SQL	Structured Query Language
ORM	Object-Relational Mapping
MVC	Model View Controller
DTO	Data Transfer Object
CRUD	Create Read Update Delete
OAuth	Open Authorization
UML	Unified Modeling Language
URL	Uniform Resource Locator
UI	User Interface
UX	User Experience
AI	Artificial Intelligence
NLP	Natural Language Processing
LLM	Large Language Model
CV	Curriculum Vitae
PFA	Projet de Fin d'Année
SOLID	Single Responsibility, Open/Closed , Liskov Substitution , Interface Segregation, and Dependency Inversion Principle
DRY	Don't Repeat Yourself
KISS	Keep It Simple Stupid

Table des matières

Dédicace **i**

Remerciements **ii**

Résumé **iv**

Abstract **v**

vi **ملخص**

Liste des abréviations **vii**

Table des figures **ix**

Liste des tableaux **x**

Introduction **1**

1 Project Management App **4**

1.1 Contexte du projet 4

1.1.1 Description des besoins 4

1.1.2 Besoins fonctionnels 4

1.1.3 Les besoins non-fonctionnels 4

1.2 Analyse et modélisation 4

1.2.1 Importance de la modélisation 4

1.2.2 Définition UML 5

1.2.3 Diagramme de use case 5

1.2.4 Diagrammes de séquence 5

1.2.5 Diagramme de classes 5

1.3 Réalisation 6

1.3.1 Méthode de gestion 6

1.3.2 Environnement de programmation 6

1.3.3 Bonnes pratiques et assurance des BNF 11

1.3.4 Interfaces de l'application 13

2 CV Builder App **20**
2.1 Contexte du projet . . . 20
2.1.1 Description des besoins . . . 20
2.1.2 Besoins fonctionnels . . . 20
2.1.3 Les besoins non-fonctionnels . . . 20
2.2 Analyse et modélisation . . . 20
2.2.1 Importance de la modélisation . . . 20
2.2.2 Définition UML . . . 21
2.2.3 Diagramme de use case . . . 21
2.2.4 Diagrammes de séquence . . . 21
2.2.5 Diagramme de classes . . . 21
2.3 Réalisation . . . 22
2.3.1 Méthode de gestion . . . 22
2.3.2 Environnement de programmation . . . 22
2.3.3 Bonnes pratiques et assurance des BNF . . . 23

3 Algo App **25**
3.1 Contexte du projet . . . 25
3.1.1 Description des besoins . . . 25
3.1.2 Besoins fonctionnels . . . 25
3.1.3 Les besoins non-fonctionnels . . . 25
3.2 Analyse et modélisation . . . 25
3.2.1 Importance de la modélisation . . . 25
3.2.2 Définition UML . . . 26
3.2.3 Diagramme de use case . . . 26
3.2.4 Diagrammes de séquence . . . 26
3.2.5 Diagramme de classes . . . 26
3.3 Réalisation . . . 27
3.3.1 Méthode de gestion . . . 27
3.3.2 Environnement de programmation . . . 27
3.3.3 Bonnes pratiques et assurance des BNF . . . 27

4 Entretien App **29**
4.1 Contexte du projet . . . 29
4.1.1 Description des besoins . . . 29
4.1.2 Besoins fonctionnels . . . 29
4.1.3 Les besoins non-fonctionnels . . . 29
4.2 Analyse et modélisation . . . 29
4.2.1 Importance de la modélisation . . . 29
4.2.2 Définition UML . . . 30

4.2.3 Diagramme de use case 30
4.2.4 Diagrammes de séquence 30
4.2.5 Diagramme de classes 30
4.3 Réalisation 31
4.3.1 Méthode de gestion 31
4.3.2 Environnement de programmation 31
4.3.3 Bonnes pratiques et assurance des BNF 31

Conclusion **32**

Webographie **33**

Annexe **35**

Table des figures

Chapitre 1 – Project Management App

1.1 Page des environnements de travail 13
1.2 Dashboard 14
1.3 Page de gestion des tickets 14
1.4 Page de gestion des équipes 15
1.5 Centre de notifications en temps réel 16
1.6 Vue calendrier avec planification de réunions 16
1.7 Espace de discussions collaboratif 17
1.8 Journal des activités avec filtrage avancé 18

Chapitre 2 – CV Builder App

Chapitre 3 – Algo App

Chapitre 4 – Entretien App

Liste des tableaux

Chapitre 1 – Project Management App

1.1 Outils de développement utilisés 7
1.2 Langages de programmation utilisés 8
1.3 Frameworks utilisés 9
1.4 Bibliothèques utilisées 10
1.5 Technologies d'infrastructure 11

Chapitre 2 – CV Builder App

Chapitre 3 – Algo App

Chapitre 4 – Entretien App

Introduction

Le domaine de l'ingénierie informatique connaît une transformation profonde, portée par l'essor des technologies numériques, de l'intelligence artificielle et des architectures logicielles distribuées. Dans un contexte où les entreprises recherchent des profils polyvalents, capables de concevoir, développer et déployer des solutions complexes, les établissements de formation se doivent de préparer leurs étudiants à relever ces défis avec rigueur et créativité. L'École des Hautes Études d'Ingénierie d'Oujda, fidèle à sa vocation d'excellence, inscrit le projet de fin d'année comme un exercice fondamental permettant aux étudiants de mobiliser l'ensemble de leurs acquis académiques dans un cadre concret et professionnalisant.

Cependant, au-delà des compétences purement techniques, le passage du monde académique au monde professionnel exige la maîtrise de dimensions souvent négligées dans les cursus traditionnels : la gestion efficace de projets collaboratifs, le perfectionnement des aptitudes en résolution de problèmes algorithmiques, la construction d'un profil professionnel attractif, ou encore la préparation méthodique aux processus de recrutement. Ces compétences transversales, bien qu'essentielles, sont rarement adressées de manière intégrée par les outils existants, laissant les étudiants face à un écosystème fragmenté de ressources dispersées et peu adaptées à leurs besoins spécifiques.

C'est précisément à partir de ce constat, nourri par notre propre vécu en tant qu'étudiants en quatrième année de Génie Informatique, que l'idée de ce projet a émergé. Notre ambition est de concevoir et de développer une plateforme unifiée qui rassemble, au sein d'un même environnement, plusieurs outils complémentaires destinés à accompagner l'étudiant ingénieur tout au long de son parcours. Cette plateforme intégrée s'articule autour de quatre modules principaux :

- Une application dédiée à la gestion de projets collaboratifs.
- Une application orientée vers la création de CV et de portfolios ainsi que la recherche de stages et d'opportunités professionnelles.
- Une application consacrée à l'entraînement en algorithmique et à la résolution de problèmes.
- Une application dédiée à la préparation aux entretiens techniques et comportementaux.

Le présent rapport retrace l'ensemble de la démarche adoptée pour mener à bien ce projet ambitieux. Il débute par une analyse du contexte et des besoins identifiés, avant de présenter les

choix architecturaux et technologiques retenus. Les différentes composantes de la plateforme y seront détaillées à tour de rôle tout au long du document. Les phases de conception et de réalisation y sont ensuite exposées de manière méthodique, suivies d'une présentation des résultats obtenus. Enfin, une réflexion sur les perspectives d'évolution de la solution clôture ce document, témoignant de notre volonté de pérenniser cet outil au service de la communauté étudiante.

Chapitre 1

Project Management App

CHAPITRE 1

Project Management App

1.1 Contexte du projet

1.1.1 Description des besoins

La plateforme de gestion de projets répond à un besoin essentiel : centraliser les outils de collaboration et de gestion de tâches pour les équipes d'étudiants. Les utilisateurs requièrent une solution intuitive permettant de gérer des projets complexes avec plusieurs membres, des workflows personnalisables et une communication en temps réel.

1.1.2 Besoins fonctionnels

Les besoins fonctionnels incluent : l'authentification sécurisée, la gestion des utilisateurs, la création d'environnements collaboratifs, la gestion des équipes, la création et suivi des tickets, les notifications en temps réel, et la journalisation complète des actions. Au total, 82 besoins fonctionnels ont été identifiés et documentés.

1.1.3 Les besoins non-fonctionnels

La plateforme doit assurer : la performance optimale via Apache Kafka, la sécurité avec hachage cryptographique et gestion JWT, la scalabilité via microservices, la disponibilité avec redémarrage automatique, la maintenabilité avec architecture en couches, la portabilité Docker, l'interopérabilité REST/JSON et WebSocket, ainsi que l'observabilité avec Serilog et Seq.

1.2 Analyse et modélisation

1.2.1 Importance de la modélisation

La modélisation est cruciale pour structurer la complexité du projet, communiquer efficacement avec les équipes et valider l'architecture avant implémentation.

1.2.2 Définition UML

UML (Unified Modeling Language) est un standard de notation graphique permettant de visualiser et documenter les systèmes logiciels complexes.

1.2.3 Diagramme de use case

Définition

Un diagramme de use case représente les interactions entre les acteurs (utilisateurs ou systèmes externes) et le système, illustrant les fonctionnalités principales du point de vue utilisateur.

Notre diagramme de use case

Le diagramme de use case du Project Management App inclut les acteurs : Utilisateur Anonyme, Utilisateur Authentifié, Chef de Projet et Administrateur. Les use cases couvrent l'authentification, la gestion des environnements, la gestion des équipes, la gestion des tickets et des notifications.

1.2.4 Diagrammes de séquence

Définition

Un diagramme de séquence illustre l'ordre des interactions entre objets dans le temps, montrant comment les composants communiquent pour accomplir une tâche.

Nos diagrammes de séquence

Les principaux diagrammes de séquence documentent : l'authentification utilisateur, la création d'un projet, l'assignation d'une tâche, la notification en temps réel et la mise à jour du statut d'un ticket.

1.2.5 Diagramme de classes

Définition

Un diagramme de classes représente la structure statique du système, montrant les classes, leurs attributs, méthodes et relations.

Notre diagramme de classes

Le modèle inclut les entités : User, Environment, Team, Ticket, Workflow, WorkflowStep, Notification, et Log, avec leurs relations d'association, d'héritage et de composition.

1.3 Réalisation

1.3.1 Méthode de gestion

Définition

La méthode de gestion de projet définit le cadre organisationnel qui structure la planification, le suivi et la livraison des fonctionnalités. Elle détermine comment les tâches sont réparties entre les membres de l'équipe, comment les priorités sont établies, et comment les itérations sont rythmées pour assurer une progression continue et mesurable du projet.

De la théorie à la pratique

Pour le développement du Project Management App, nous avons adopté une approche hybride combinant **Scrum** et **Kanban**. Le projet a été découpé en sprints de deux semaines, avec des *daily standups* pour synchroniser l'avancement de chaque microservice. Le suivi des tâches s'est effectué sur un tableau Kanban à quatre colonnes : `To Do`, `Doing`, `Done` et `Happy Day`, reflétant directement les statuts de tickets implémentés dans l'application elle-même. Chaque sprint visait la livraison d'un service complet — de la couche données à la couche présentation — permettant une intégration incrémentale. Les revues de sprint ont servi à valider les fonctionnalités avec l'encadrant et à ajuster les priorités du backlog.

1.3.2 Environnement de programmation

Outils utilisés

Le tableau 1.1 présente les outils de développement utilisés pour la réalisation du Project Management App.

Outil	Logo	Description
Visual Studio Code		Éditeur de code principal utilisé pour le développement frontend Angular et backend .NET, avec extensions pour C#, TypeScript, Docker et Git.
Docker & Docker Compose		Conteneurisation de l'ensemble des microservices, de la base de données SQL Server, de Kafka et de Zookeeper via des fichiers `docker-compose.yaml` multi-services.
Git & GitHub		Contrôle de version distribué avec gestion des branches par fonctionnalité et revues de code via pull requests.
Postman		Test et validation des endpoints REST de chaque microservice, avec collections organisées par service.

TAB. 1.1: Outils de développement utilisés

Langages utilisés

Le tableau 1.2 présente les langages de programmation employés dans le projet.

Langage	Logo	Utilisation
C#		Langage principal pour la totalité de microservices backend (Logs-Service, Notifications-Service, Teams-Service, etc), exploitant la programmation orientée objet, les génériques et la programmation asynchrone (`async/await`).
TypeScript		Langage fortement typé utilisé pour le développement de l'application frontend Angular 20, offrant la sûreté de type et l'autocomplétion pour une base de code maintenable.
SQL		Utilisé via Entity Framework Core pour les migrations de schéma et les requêtes LINQ traduites vers SQL Server pour les services Logs et Teams.
HTML / CSS		Structuration des templates Angular et stylisation avec Tailwind CSS 3.4 pour une interface responsive et moderne avec un thème sombre.

TAB. 1.2: Langages de programmation utilisés

Frameworks utilisés

Le tableau 1.3 détaille les frameworks structurants du projet.

Framework	Version	Rôle
ASP.NET Core	9.0	Framework backend pour la construction des APIs REST des microservices.
Angular	20.3	Framework frontend pour le développement de l'application SPA (Single Page Application). Utilise l'architecture *standalone components*, les *signals* pour la gestion d'état réactive, et le routage déclaratif.
Entity Framework Core	9.0	ORM (Object-Relational Mapping) pour l'accès aux données SQL Server, la génération de migrations et l'exécution de requêtes LINQ avec stratégies `AsNoTracking` pour les lectures.
Tailwind CSS	3.4.18	Framework CSS utilitaire pour le stylisme rapide de l'interface utilisateur avec un système de couleurs personnalisé (`card`, `cardBorder`, `cardText`) et un thème sombre cohérent.

TAB. 1.3: Frameworks utilisés

Bibliothèques utilisées

Le tableau 1.4 recense les bibliothèques tierces intégrées au projet.

Bibliothèque	Version	Rôle
Confluent.Kafka	2.3 / 2.12	Client .NET pour Apache Kafka, utilisé pour la production et la consommation d'événements asynchrones entre les microservices (création d'équipe, assignation de tâche, etc.).
SignalR	intégré	Bibliothèque de communication temps réel via WebSockets, utilisée dans le Notifications-Service pour diffuser les notifications instantanément aux clients connectés via le hub `/notificationHub`.
Swashbuckle	6.5 / 7.2 / 10.0	Génération automatique de la documentation OpenAPI/Swagger pour chaque microservice, fournissant une interface de test interactive.
Newtonsoft.Json	13.0.3	Sérialisation et désérialisation JSON avancée, utilisée dans le Notifications-Service pour la persistance fichier des notifications.
Lucide Angular	0.554.0	Bibliothèque d'icônes SVG pour Angular, fournissant les icônes de l'interface (recherche, calendrier, alertes, utilisateurs, etc.).
RxJS	7.8.0	Bibliothèque de programmation réactive pour la gestion des flux de données asynchrones côté frontend (appels HTTP, événements SignalR, filtrage en temps réel).

Tab. 1.4: Bibliothèques utilisées

Technologies utilisées

Le tableau 1.5 présente les technologies d'infrastructure utilisées.

Technologie	Rôle dans le projet
SQL Server 2022	Base de données relationnelle pour les services Logs et Teams. Chaque service possède sa propre base (`LogsDB`, `TeamsDB`) avec des schémas isolés et des index optimisés (timestamp, service, userId, level).
Apache Kafka 7.5	Plateforme de streaming d'événements assurant la communication asynchrone entre microservices. Gère plus de 10 topics incluant : `team-created`, `team-updated`, `team-deleted`, `team.member.added`, `team.member.removed`, `task.assigned`, `meeting.scheduled`, `ticket.note.added`, etc.
Apache Zookeeper	Service de coordination pour le cluster Kafka, gérant la découverte des brokers et la synchronisation des métadonnées (port 2181).
Docker	Conteneurisation de chaque microservice avec des Dockerfiles multi-étapes (restore → build → publish → runtime) et exécution en tant qu'utilisateur non-root pour la sécurité.
JWT	JSON Web Tokens pour l'authentification. Les tokens sont stockés côté client dans le `localStorage` et injectés automatiquement via un intercepteur HTTP Angular (`authInterceptor`).
WebSocket (SignalR)	Protocole de communication bidirectionnelle pour les notifications temps réel. Le hub `/notificationHub` diffuse les événements `ReceiveNotification`, `NotificationUpdated`, `AllNotificationsRead` et `NotificationDeleted` aux clients.

TAB. 1.5: Technologies d'infrastructure

1.3.3 Bonnes pratiques et assurance des BNF

La qualité logicielle du Project Management App repose sur l'application rigoureuse de plusieurs bonnes pratiques architecturales et de développement, assurant simultanément le respect des besoins non-fonctionnels identifiés lors de la phase d'analyse.

Architecture en couches et pattern Repository

Chaque microservice adopte une architecture en couches clairement séparées : **Controllers** (couche présentation), **Services** (couche métier) et **Repositories** (couche accès aux données). Le pattern Repository isole la logique d'accès aux données derrière des interfaces, facilitant les tests et permettant de changer de source de données sans impacter la logique métier. Par exemple, le Notifications-Service utilise un repository basé sur un fichier JSON avec verrouillage par `SemaphoreSlim`, tandis que les services Logs et Teams utilisent Entity Framework Core avec SQL Server.

Injection de dépendances et DTOs

L'injection de dépendances est systématiquement utilisée dans les trois services backend, conformément au conteneur IoC natif d'ASP.NET Core. Les DTOs (Data Transfer Objects) séparent strictement les modèles internes des contrats d'API : chaque entité possède ses `CreateDto`, `UpdateDto` et `ResponseDto`, avec AutoMapper assurant la conversion automatique.

Gestion des erreurs et intercepteurs

Côté frontend, un **intercepteur d'erreurs HTTP** capture toutes les réponses d'erreur et les transforme en messages utilisables, couvrant les codes 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 409 (Conflict), 422 (Validation Error), 500 (Server Error), 502 (Bad Gateway), 503 (Service Unavailable) et 504 (Gateway Timeout). Un **intercepteur d'authentification** injecte automatiquement le token JWT dans chaque requête sortante.

Sécurité

La sécurité est assurée à plusieurs niveaux : authentification JWT avec injection automatique via intercepteur, configuration CORS stricte n'autorisant que l'origine `http://localhost:4200` avec credentials, exécution des conteneurs Docker en tant qu'utilisateur non-root (`appuser`), et commits manuels Kafka après traitement réussi pour garantir la fiabilité de la messagerie (`EnableAutoCommit: false`).

Résilience et santé des services

Les services intègrent des mécanismes de résilience : les connexions Entity Framework Core sont configurées avec une stratégie de *retry on failure* (5 tentatives, délai de 30 secondes), et des *health checks* surveillent la disponibilité de SQL Server et du DbContext. Les migrations de base de données s'exécutent automatiquement au démarrage de chaque service.

1.3.4 Interfaces de l'application

Cette sous-section présente les principales interfaces utilisateur du Project Management App, illustrant les fonctionnalités implémentées.

Page des environnements

La page d'accueil de l'application affiche les environnements auxquels l'utilisateur participe sous forme de cartes dans une grille responsive. Elle propose la création de nouveaux environnements et la consultation des invitations reçues via des modales dédiées. Un état vide avec une illustration guide les nouveaux utilisateurs.

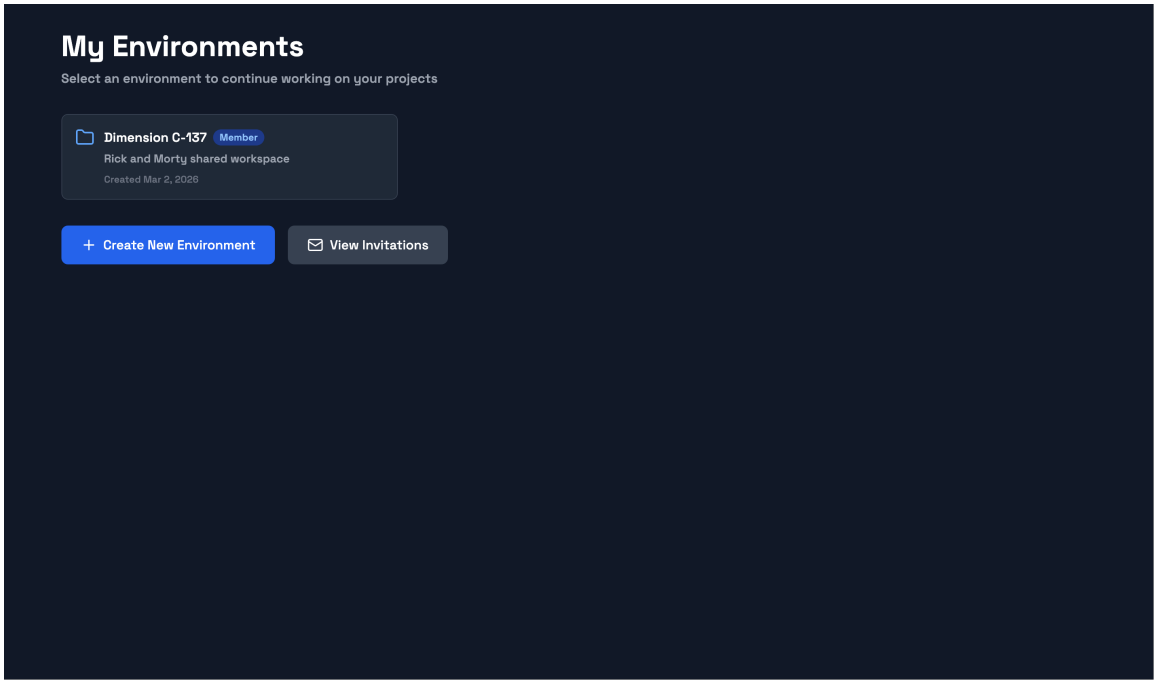

FIG. 1.1: Page des environnements de travail

Tableau de bord (Dashboard)

Le tableau de bord offre une vue d'ensemble du projet avec trois zones principales : une section de synthèse en haut, un tableau Kanban de type *drag-and-drop* affichant les tickets de l'utilisateur connecté (colonnes `To Do`, `Doing`, `Done`, `Happy Day`) et une barre latérale montrant les réunions à venir et les discussions récentes.

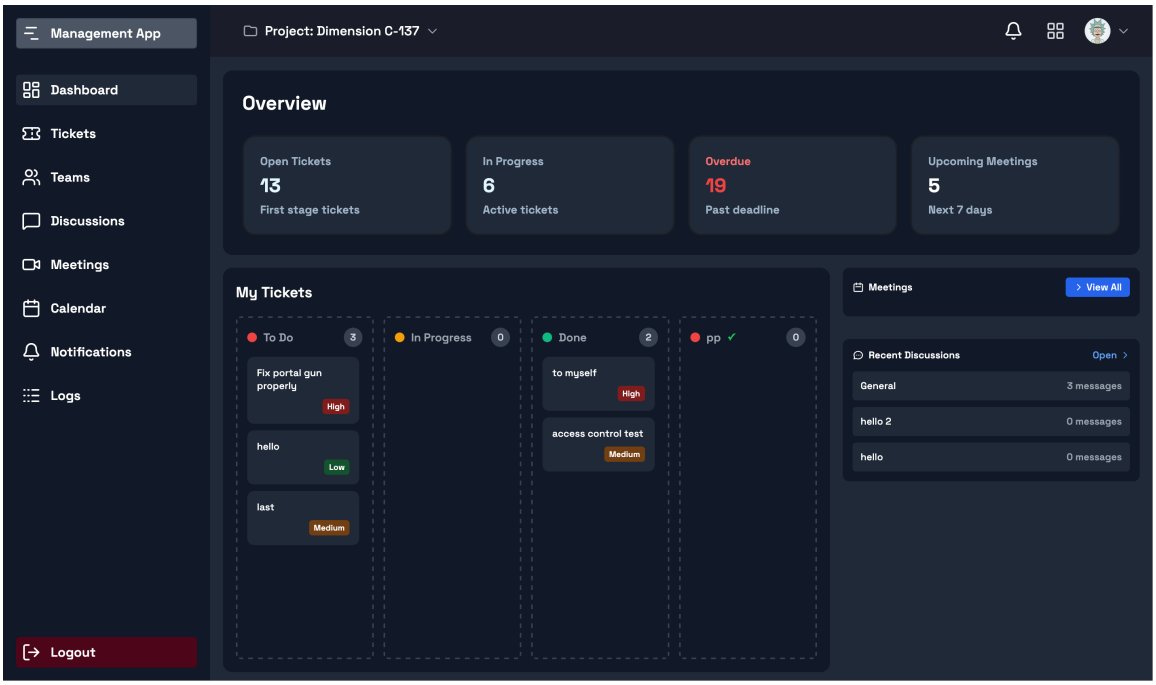

FIG. 1.2: Dashboard

Gestion des tickets

La page de gestion des tickets propose un tableau Kanban plein écran avec une barre d'outils permettant le filtrage par recherche textuelle, par priorité (Low, Medium, High) et par assigné. La création de tickets s'effectue via une modale permettant de définir le titre, le statut, la priorité et les membres assignés.

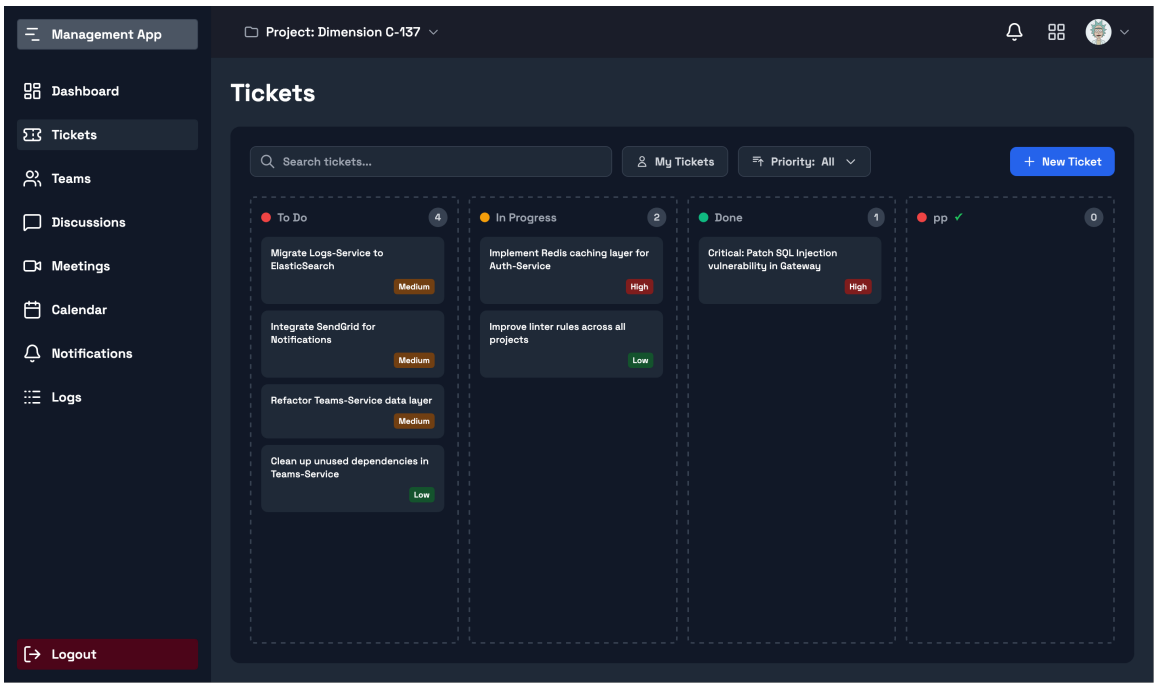

FIG. 1.3: Page de gestion des tickets

Gestion des équipes

La page des équipes affiche les équipes sous forme de cartes avec recherche, tri (ascendant/descendant) et possibilité de création via une modale. Chaque carte d'équipe présente les détails de l'équipe et permet d'accéder à une vue détaillée pour la gestion des membres.

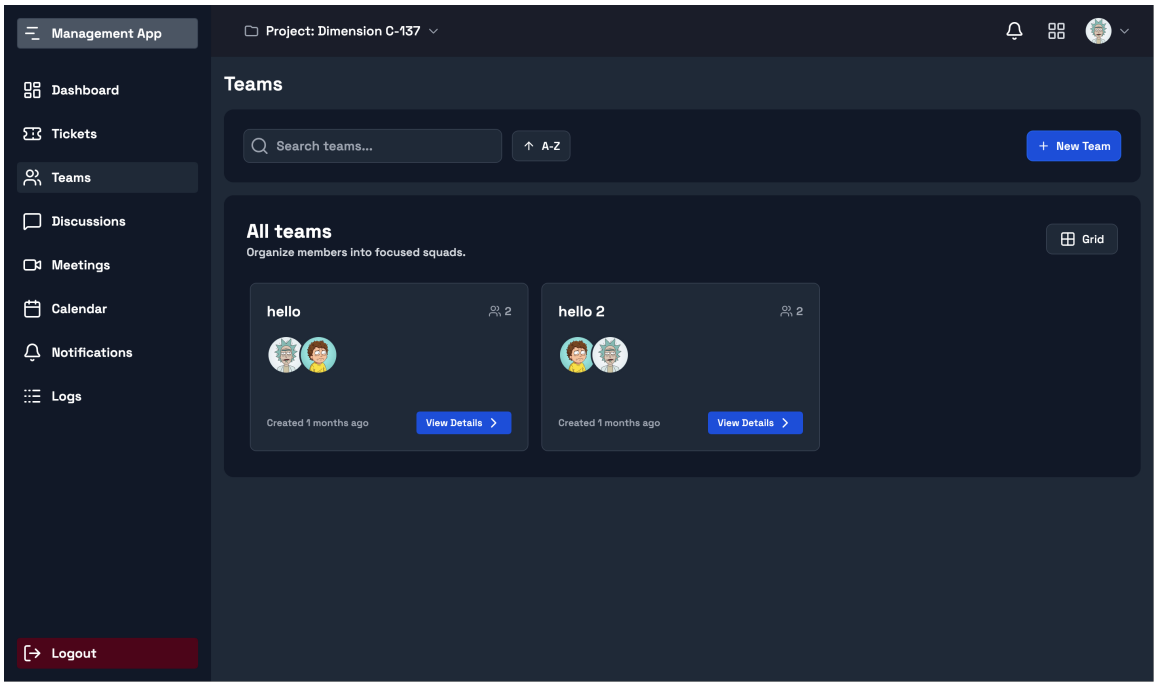

FIG. 1.4: Page de gestion des équipes

Centre de notifications

Le centre de notifications adopte une disposition en deux volets : une boîte de réception (70% de la largeur) avec filtrage par type (Tous, Mentions, Tâches, Réunions) et un panneau de détail/statistiques (30%). Les notifications sont reçues en temps réel via SignalR et offrent des actions de lecture, archivage et suppression.

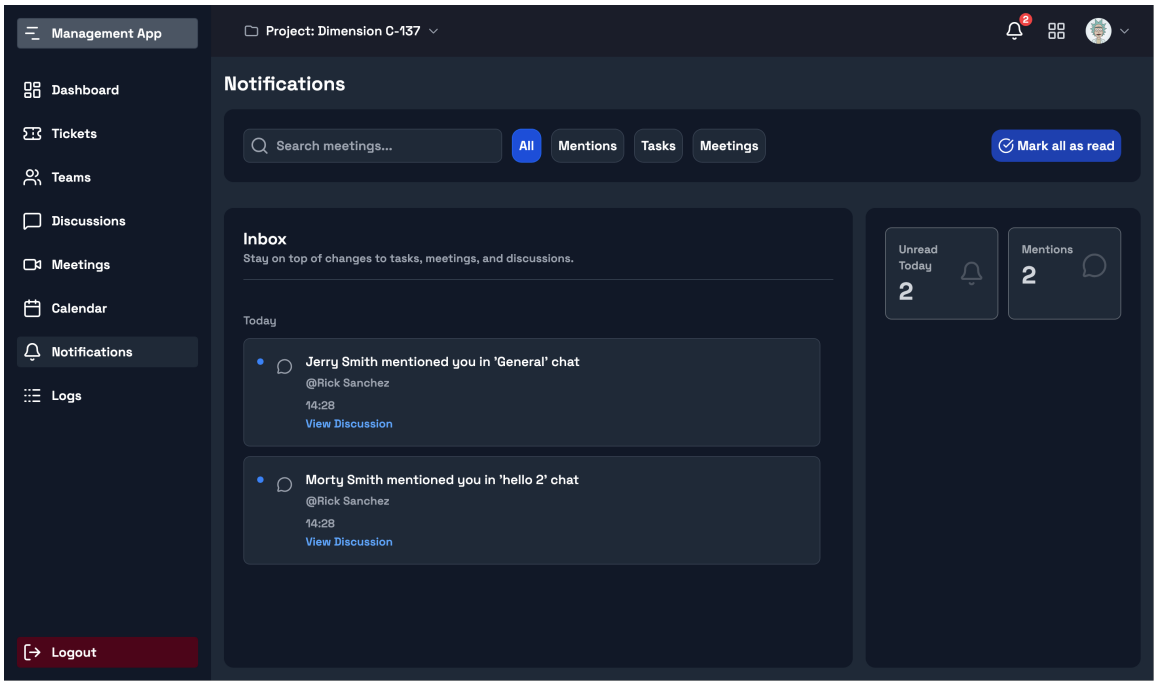

FIG. 1.5: Centre de notifications en temps réel

Calendrier et réunions

Le module calendrier supporte trois vues : semaine (par défaut), jour et mois. La barre d'outils permet la navigation temporelle, la recherche et la planification de nouvelles réunions. La page des réunions dédiée offre un filtrage avancé (Aujourd'hui, Cette semaine, Mes réunions) avec un panneau de détail latéral.

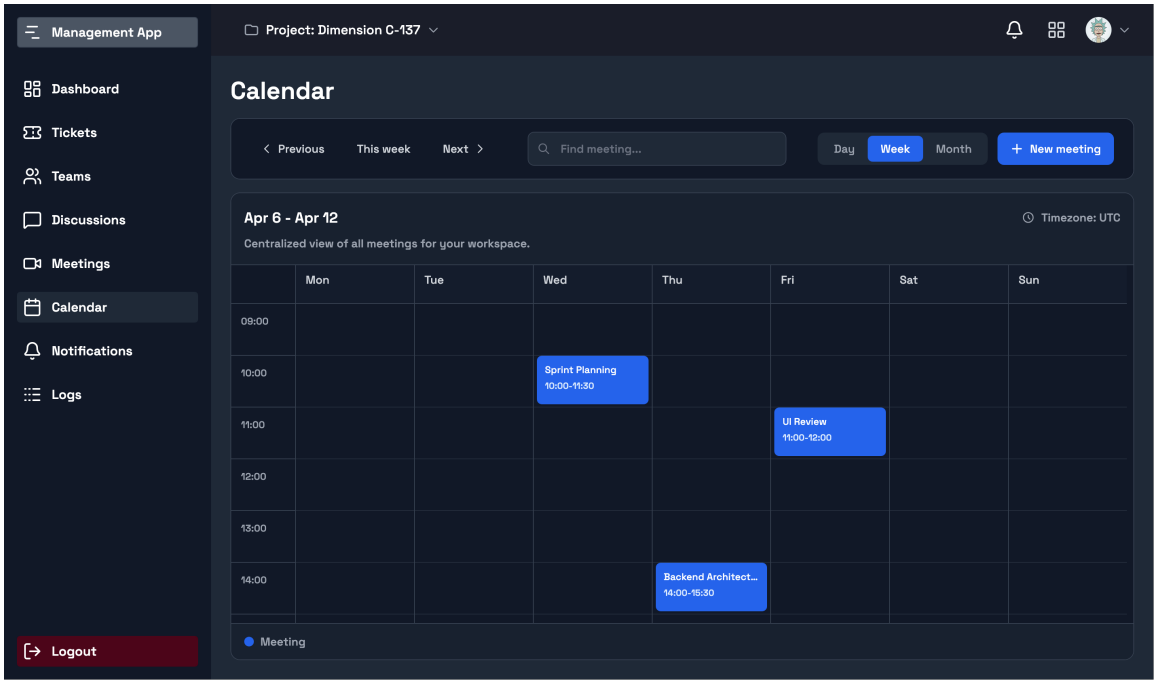

FIG. 1.6: Vue calendrier avec planification de réunions

Espace de discussions

L'espace de discussions présente une disposition en deux colonnes : une liste de conversations (35%) et une zone de chat détaillée (65%). Chaque conversation affiche les avatars des participants, les messages avec horodatage, les pièces jointes avec icônes et taille, ainsi que les réactions sous forme d'emojis.

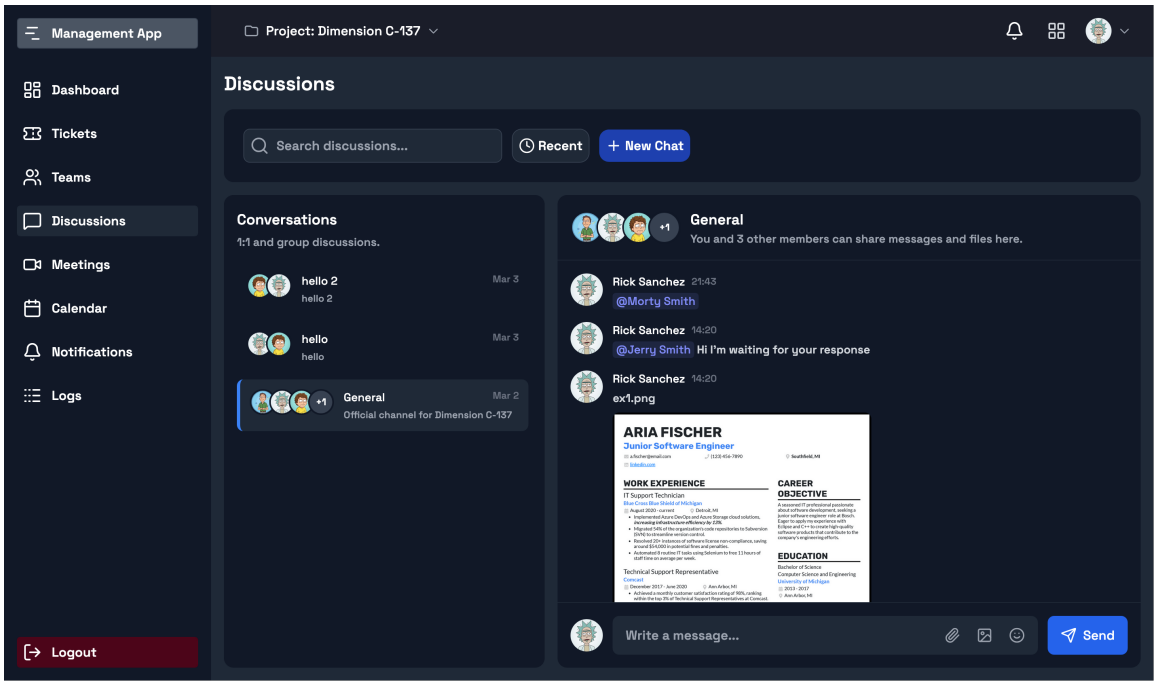

FIG. 1.7: Espace de discussions collaboratif

Journal des activités (Logs)

La page des logs présente quatre cartes de statistiques en haut (total, erreurs, avertissements, logs du jour), suivies de filtres avancés (plage de dates, service, niveau, recherche) avec export, et d'un tableau paginé de 50 entrées par page affichant l'ensemble des activités du système.

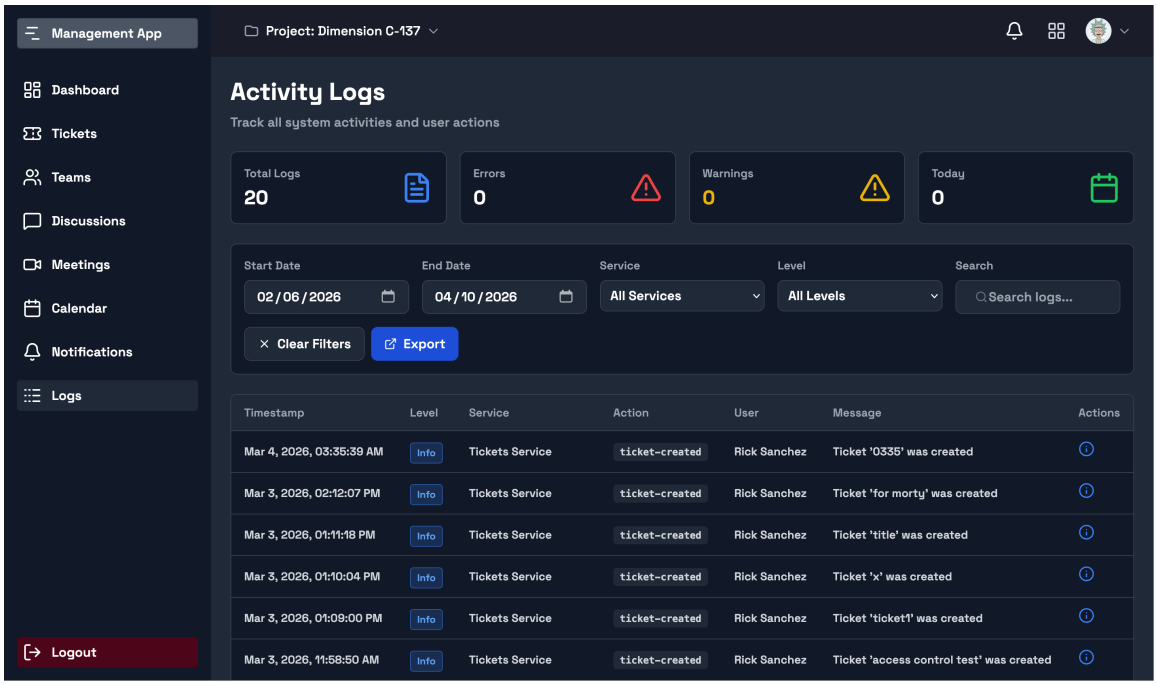

FIG. 1.8: Journal des activités avec filtrage avancé

Chapitre 2

CV Builder App

CHAPITRE 2

CV Builder App

2.1 Contexte du projet

2.1.1 Description des besoins

La plateforme de création et d'optimisation de CV répond à un besoin critique : aider les étudiants à construire un profil professionnel attractif et à identifier les opportunités d'emploi correspondant à leur profil. Les utilisateurs demandent des outils simples pour créer un CV, analyser sa qualité et matcher avec des offres d'emploi.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels incluent : création de CV avec templates, génération de portfolio, analyse d'offres d'emploi, matching intelligent CV/offres, analyse linguistique et structurelle du CV, suggestions d'amélioration et calcul de score de qualité. Au total, 23 besoins fonctionnels ont été identifiés.

2.1.3 Les besoins non-fonctionnels

La plateforme doit assurer : extraction de données par IA, performance optimale pour la génération de documents, sécurité des données personnelles, scalabilité pour gérer une base croissante de CV et d'offres, disponibilité constante, maintenabilité du code, portabilité Docker et interopérabilité avec LinkedIn.

2.2 Analyse et modélisation

2.2.1 Importance de la modélisation

La modélisation permet de structurer les flux de données complexes entre les services CV Creator, Portfolio Generator, Job Matcher et Analyzer, garantissant une cohérence globale.

2.2.2 Définition UML

UML fournit la notation formelle pour documenter l'interaction entre les cinq microservices et les flux de génération de CV.

2.2.3 Diagramme de use case

Définition

Le diagramme de use case montre comment un utilisateur interagit avec la plateforme pour créer, analyser et matcher son CV avec des offres.

Notre diagramme de use case

Les acteurs sont : Étudiant, Recruteur. Les use cases incluent : créer un CV, sélectionner un template, générer un portfolio, analyser un CV, matcher avec des offres, consulter les offres filtrées.

2.2.4 Diagrammes de séquence

Définition

Un diagramme de séquence représente l'ordre chronologique des appels entre les cinq services lors d'une opération donnée.

Nos diagrammes de séquence

Les diagrammes documentent : la création de CV (formulaire → création → stockage), l'analyse de CV (upload → analyse IA → score), le matching (CV → analyse → offres → score de compatibilité).

2.2.5 Diagramme de classes

Définition

Le diagramme de classes montre la structure des entités : CV, Portfolio, JobOffer, CVAnalysis, MatchResult et leurs relations.

Notre diagramme de classes

Les classes principales incluent CV (avec sections), Template, Portfolio (avec liens), JobOffer (parsing LinkedIn), CVAnalysisResult (score, suggestions), et MatchResult (score technique/soft skills/expérience).

2.3 Réalisation

2.3.1 Méthode de gestion

Définition

Organisation itérative du développement avec des points de contrôle sur chaque service.

De la théorie à la pratique

Sprints de deux semaines focalisés sur chaque service, intégration progressive, tests d'API et validation avec des utilisateurs réels.

2.3.2 Environnement de programmation

Outils utilisés

Visual Studio Code, Docker Compose, Git, Postman, Insomnia pour tester les APIs de CV, iTextSharp (ou DinkToPdf) pour la génération PDF.

Langages utilisés

C# pour les services, TypeScript/Angular pour le frontend, Python pour l'extraction de données LinkedIn.

Frameworks utilisés

ASP.NET Core pour les services backend, Angular pour le frontend, Selenium pour le scraping LinkedIn.

Bibliothèques utilisées

AutoMapper, Serilog, OpenAI API (ou Hugging Face) pour l'analyse d'IA, iTextSharp pour PDF generation, Newtonsoft.Json pour la sérialisation.

Technologies utilisées

PostgreSQL pour persister les CV et offres, Redis pour cacher les offres d'emploi, Python FastAPI pour le service d'IA, Kafka pour événements de matching.

2.3.3 Bonnes pratiques et assurance des BNF

Utilisation de DTOs pour isoler les formats API, validation stricte des entrées, chiffrement des données sensibles, tests de performance pour les analyses IA, documentation Swagger complète.

Chapitre 3

Algo App

CHAPITRE 3

Algo App

3.1 Contexte du projet

3.1.1 Description des besoins

L'application d'entraînement en algorithmique répond au besoin fondamental : préparer les étudiants aux entretiens techniques des grandes entreprises. Les utilisateurs recherchent une plateforme proposant des problèmes variés, des solutions expliquées et un système de progression.

3.1.2 Besoins fonctionnels

Les besoins fonctionnels incluent : catalogue de problèmes algorithmiques classés par difficultés, éditeur de code intégré, exécution et test de code, solutions optimales avec explications, suivi de progression, leaderboard et badges. Des fonctionnalités avancées incluent la détection de patterns, statistiques détaillées et suggestions de problèmes similaires.

3.1.3 Les besoins non-fonctionnels

La plateforme doit assurer : exécution sécurisée et isolée du code utilisateur (sandboxing), performance optimale même avec des entrées volumineuses, scalabilité pour des milliers d'utilisateurs simultanés, disponibilité 24/7, maintenabilité du code éducatif, portabilité et sécurité contre les injections de code.

3.2 Analyse et modélisation

3.2.1 Importance de la modélisation

La modélisation est essentielle pour structurer les flux complexes d'exécution de code, résultats de test et suivi de progression.

3.2.2 Définition UML

UML permet de visualiser les relations entre le système d'apprentissage, l'éditeur de code, le compilateur/interprète et le moteur de test.

3.2.3 Diagramme de use case

Définition

Le diagramme de use case visualise l'interaction entre l'apprenant et la plateforme algorithmique.

Notre diagramme de use case

Les acteurs sont : Apprenant, Instructeur, Système d'Évaluation. Les use cases incluent : consulter les problèmes, coder une solution, tester le code, visualiser la solution, consulter les statistiques, participer au leaderboard.

3.2.4 Diagrammes de séquence

Définition

Un diagramme de séquence montre le flux d'exécution lors de la soumission et l'évaluation d'une solution.

Nos diagrammes de séquence

Les diagrammes documentent : soumettre une solution (éditeur → backend → compilateur → test → feedback), consulter le leaderboard (filtrage → tri → affichage), voir les explications (analyse de code → suggestions de patterns).

3.2.5 Diagramme de classes

Définition

Le diagramme de classes représente les entités principales du système algorithmique.

Notre diagramme de classes

Les classes incluent : Problem, TestCase, Solution, User, ExecutionResult, Leaderboard, Badge, avec leurs attributs et relations.

3.3 Réalisation

3.3.1 Méthode de gestion

Définition

Gestion structurée des problèmes, des cas de test et de l'évolution de la plateforme.

De la théorie à la pratique

Ajout progressif de problèmes par difficulté, tests exhaustifs des cas limites, feedback utilisateur intégré au processus d'amélioration.

3.3.2 Environnement de programmation

Outils utilisés

Visual Studio Code, Docker pour isoler les exécutions, Git, Jenkins pour l'intégration continue, Prometheus pour le monitoring.

Langages utilisés

C# pour le backend, TypeScript pour le frontend, Python/Java/C++ (runtime sandboxé) pour l'exécution des solutions utilisateurs.

Frameworks utilisés

ASP.NET Core, Angular, Docker avec namespace isolation pour le sandboxing.

Bibliothèques utilisées

Docker SDK, Roslyn (analyse C#), Judge0 ou similaire pour l'exécution sécurisée, xUnit pour les tests unitaires.

Technologies utilisées

PostgreSQL pour les problèmes et solutions, Redis pour le cache des leaderboards, Docker pour le sandboxing, Kubernetes pour la scalabilité.

3.3.3 Bonnes pratiques et assurance des BNF

Sandboxing strict du code utilisateur, timeouts et limite mémoire, tests exhaustifs des cas limites, monitoring continu des performances, documentation claire des algorithmes et patterns.

Chapitre 4

Entretien App

CHAPITRE 4

Entretien App

4.1 Contexte du projet

4.1.1 Description des besoins

L'application de préparation aux entretiens répond à un besoin crucial : préparer les étudiants aux entretiens techniques et comportementaux menés par les grandes entreprises. Les utilisateurs demandent des simulations d'entretiens, des feedbacks personnalisés et des ressources de développement personnel.

4.1.2 Besoins fonctionnels

Les besoins fonctionnels incluent : banque de questions d'entretien, simulations d'entretiens en temps réel, enregistrement et relecture vidéo, analyse de la performance, feedbacks détaillés, suggestions d'amélioration, ressources par domaine technique et soft skills.

4.1.3 Les besoins non-fonctionnels

La plateforme doit assurer : qualité vidéo optimale, latence minimale pour les simulations en temps réel, analyse du langage naturel pour le feedback, scalabilité pour les sessions simultanées, disponibilité constante, maintenabilité du contenu éducatif, portabilité sur tous les navigateurs et appareils.

4.2 Analyse et modélisation

4.2.1 Importance de la modélisation

La modélisation structure les flux complexes d'enregistrement vidéo, analyse de performance et génération de feedbacks.

4.2.2 Définition UML

UML visualise les interactions entre le système de simulation, le moteur d'analyse et le service de feedback.

4.2.3 Diagramme de use case

Définition

Le diagramme de use case montre comment un candidat interagit avec la plateforme pour s'entraîner aux entretiens.

Notre diagramme de use case

Les acteurs sont : Candidat, Mentor Virtuel, Analyseur de Performance. Les use cases incluent : consulter les questions, pratiquer une simulation, enregistrer une réponse, recevoir un feedback, consulter les ressources, suivre la progression.

4.2.4 Diagrammes de séquence

Définition

Un diagramme de séquence illustre le flux d'une simulation d'entretien de bout en bout.

Nos diagrammes de séquence

Les diagrammes documentent : démarrer une simulation (sélection question → enregistrement vidéo → traitement → analyse → feedback), lister les ressources (filtrage → affichage organisé), suivre la progression (statistiques → comparaison).

4.2.5 Diagramme de classes

Définition

Le diagramme de classes représente les entités principales du système d'entraînement aux entretiens.

Notre diagramme de classes

Les classes incluent : Question, InterviewSimulation, VideoRecording, PerformanceAnalysis, Feedback, Resource, UserProgress, avec leurs attributs et relations.

4.3 Réalisation

4.3.1 Méthode de gestion

Définition

Gestion structurée de la création de contenu questions, des cas de test d'IA et du développement de l'analyseur.

De la théorie à la pratique

Curation de questions réalistes par domaine, raffinage progressif de l'analyseur d'IA, tests avec candidats réels, intération basée sur les résultats.

4.3.2 Environnement de programmation

Outils utilisés

Visual Studio Code, Docker, Git, WebRTC pour les communications vidéo, Jitsi ou OBS pour le streaming.

Langages utilisés

C# pour le backend, TypeScript pour le frontend, Python pour l'analyse NLP et vidéo.

Frameworks utilisés

ASP.NET Core, Angular avec WebRTC, Python FastAPI pour l'analyse.

Bibliothèques utilisées

OpenAI API (ou LLM local) pour la génération de feedbacks, TensorFlow/PyTorch pour l'analyse du langage corporel, FFmpeg pour le traitement vidéo, OpenSSL pour le chiffrement.

Technologies utilisées

PostgreSQL pour les questions et résultats, S3 ou MinIO pour le stockage vidéo, Redis pour les sessions WebRTC, Kafka pour les événements d'analyse.

4.3.3 Bonnes pratiques et assurance des BNF

Respect de la vie privée avec consentement vidéo, analyse d'IA transparente et explicable, feedback constructif et bienveillant, documentation des critères d'évaluation, tests de performance pour les sessions simultanées.

Conclusion

Ce projet de fin d'année représente l'aboutissement de quatre années d'études en Génie Informatique et l'application concrète de nos acquis académiques. Nous avons conçu et développé une plateforme intégrée composée de quatre applications complémentaires, chacune répondant à un besoin spécifique de l'étudiant ingénieur.

Le Project Management App offre une solution complète de gestion de projets collaboratifs, avec une architecture microservices robuste et scalable. Le CV Builder App aide les étudiants à construire leur profil professionnel et à identifier les opportunités d'emploi correspondantes. L'Algo App prépare les candidats aux défis techniques des entretiens d'embauche. Enfin, l'Entretien App simule les conditions d'entretien réelles et fournit des feedbacks personnalisés.

Au-delà des aspects techniques, ce projet nous a permis de développer nos compétences en gestion de projet, communication en équipe, et résolution de problèmes complexes. Nous avons appris l'importance de la modélisation, de la documentation et des bonnes pratiques de développement logiciel.

Nous envisageons de nombreuses améliorations futures : l'ajout de l'intelligence artificielle pour une analyse plus poussée, l'intégration avec les réseaux sociaux professionnels, l'expansion du catalogue de problèmes algorithmiques et de questions d'entretien, et l'amélioration de l'expérience utilisateur sur tous les appareils.

Nous remercions l'École des Hautes Études d'Ingénierie d'Oujda, notre encadrant pédagogique et technique, ainsi que tous nos camarades qui nous ont soutenus tout au long de ce projet. Cette plateforme est notre contribution à l'amélioration de la formation des futurs ingénieurs informatiques.

Webographie

Bibliographie

[1] Sam Newman. *Building Microservices*. O'Reilly Media, 2015.

[2] Microsoft Documentation. *ASP.NET Core documentation*. https ://docs.microsoft.com/en-us/aspnet/core

[3] Angular Documentation. *Angular official documentation*. https ://angular.io/docs

[4] Docker Documentation. *Docker official documentation*. https ://docs.docker.com

[5] Apache Kafka. *Apache Kafka Documentation*. https ://kafka.apache.org/documentation/

[6] Grady Booch, Robert A. Maksimchuk. *Object-Oriented Analysis and Design with Applications*. Addison-Wesley, 2007.

[7] Gang of Four. *Design Patterns : Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[8] Leonard Richardson, Mike Amundsen. *RESTful Web APIs*. O'Reilly Media, 2013.

[9] Bruce Schneier. *Cryptography Engineering*. Wiley, 2010.

[10] Ken Beck. *Extreme Programming Explained*. Addison-Wesley, 1999.

Annexe

A. Architecture globale

L'architecture de la plateforme repose sur une approche microservices avec 14 services indépendants, une base de données PostgreSQL partagée (avec schémas isolés), Redis pour le cache et la liste noire JWT, Apache Kafka pour la messagerie asynchrone, et un frontend Angular monolithique.

B. Schémas de bases de données

Chaque microservice possède son schéma logique contenant les tables essentielles. Les relations entre services sont gérées via Kafka et les APIs REST, éliminant les dépendances SQL directes.

C. Diagrammes d'architecture

Les diagrammes d'architecture système incluent la vue de déploiement Docker Compose, la vue logique des microservices, et la vue des flux de données via Kafka.

D. Exemples de code source

Des extraits de code significatifs illustrent : l'authentification JWT, le pattern Repository, la configuration Kafka, et les interceptors Angular.

E. Documentation des APIs

La documentation Swagger est disponible à `http://localhost:5000/swagger` une fois le projet déployé. Elle inclut toutes les routes, paramètres et schémas de réponse.

F. Cas de test

Les cas de test incluent : tests unitaires des services, tests d'intégration des APIs, tests de charge pour valider la performance, et tests de sécurité pour les vulnerabilités courantes.

G. Manuel de déploiement

Instructions complètes pour déployer la plateforme en environnement de développement (Docker Compose local), de staging et de production (Kubernetes ou cloud provider).

H. Calendrier du projet

Chronologie détaillée des sprints, jalons produits, et dates clés du développement sur les 6 mois du projet.

I. Résumé des technologies utilisées

Tableau récapitulatif de tous les langages, frameworks, bibliothèques et services utilisés dans la plateforme.

J. Glossaire

Définitions des termes techniques spécifiques au projet : microservices, Kafka, JWT, WebSocket, DAO, DTO, etc.